

Week 3 - Day 12a: Sorting

In this exercise, we will focus on implementing and analyzing sorting algorithms. We will start with Bubble Sort, one of the simplest sorting algorithms, and understand its time complexity.

Run the following code block first to define a helper function for testing.

```
# This function is for testing, do not modify
def check(actual, expected):
    if expected != actual:
        print(f"Failed. Function should return the value {expected},
it is returning the value {actual}")
    else:
        print("Congratulations, the test case passed!")
```

Exercise 1: Bubble Sort [] [] []

Bubble Sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The pass through the list is repeated until the list is sorted.

1.1 Implement Bubble Sort

Implement the Bubble Sort function.

```
# TODO: Implement the Bubble Sort function
def BubbleSort(lst):
    pass # Remove this line and add your code here.

# Test cases
check(BubbleSort([2, 3, 7, 1, 3, 5]), [1, 2, 3, 3, 5, 7])
check(BubbleSort([]), [])
check(BubbleSort(["b", "aaa", "addis"]), ["aaa", "addis", "b"])
```

1.2 What is the time complexity of your Bubble Sort implementation in Big-O notation?

Exercise 2: Insertion Sort [] [] []

Insertion Sort is a simple way to sort a list of items. It's not the fastest method for large lists, but it's easy to understand and works well for small lists.

Imagine you have a list of numbers that are not in order. To sort this list, you will move through the list one number at a time and put each number in the correct spot in a new, sorted list.

Here's how you do it step by step:

1. Start with initializing an empty list called `sorted_list`.
2. Take the first number from the unsorted list and put it in `sorted_list`.

3. Take the next number from the unsorted list and insert it into the right place in `sorted_list` so that the numbers in `sorted_list` remain in order.
4. Repeat this process until there are no more numbers in the unsorted list, then return the `sorted_list`

- Start with the unsorted list `L = [8, 5, 7, 1, 9, 3]`.
- Create an empty `sorted_list = []`.

```
L = [8, 5, 7, 1, 9, 3] and sorted_list = []
L = [5, 7, 1, 9, 3]   and sorted_list = [8]
L = [7, 1, 9, 3]      and sorted_list = [5, 8]
L = [1, 9, 3]          and sorted_list = [5, 7, 8]
L = [9, 3]             and sorted_list = [1, 5, 7, 8]
L = [3]                and sorted_list = [1, 5, 7, 8, 9]
L = []                 and sorted_list = [1, 3, 5, 7, 8, 9]
```

Insertion Sort

2.1 Write a function called `insertion_sort(L)` that takes a list `L` and returns a sorted list.

Hint: To insert a number into the correct spot, you can use list slicing. For example, if you have `lst = [1, 2, 4]` and you want to insert 3, you can do:

```
lst = lst[0:2] + [3] + lst[2:]
# This makes lst = [1, 2, 3, 4]

# TODO: Implement the insertion sort function
def insertion_sort(L):
    """Sorts L using insertion sort algorithm.
    Arguments: L (list of ints)
    Returns: (list of ints)
    Effects: Sorts the list
    """
    pass # Remove this line and write your solution here

# Test cases
L = [1, 4, 5, 3, 2, 7, 0]
check(insertion_sort(L), [0, 1, 2, 3, 4, 5, 7])

L = [1, 4, 5, 3, 2, 0, 7]
check(insertion_sort(L), [0, 1, 2, 3, 4, 5, 7])

L = [5, 6, 11, 12, 13]
check(insertion_sort(L), [5, 6, 11, 12, 13])

L = [12, 11, 13, 5, 6]
check(insertion_sort(L), [5, 6, 11, 12, 13])
```

Exercise 3: Selection Sort [] [] []

Selection Sort is another simple sorting algorithm. It works by repeatedly selecting the minimum element from the unsorted part of the list and moving it to the sorted part.

Imagine you have a list of numbers that are not in order. To sort this list, you will repeatedly find the smallest number in the unsorted part of the list and move it to the beginning of the list.

Here's how you do it step by step:

1. Start with the whole list being unsorted.
2. Find the smallest number in the unsorted part of the list.
3. Swap this smallest number with the first number in the unsorted part.
4. Now, the first number is sorted. Repeat the process with the rest of the list until all numbers are sorted.

- Start with the unsorted list `lst = [2, 3, 7, 1, 3, 5]`.
Here is the step-by-step process:

```
lst = [2, 3, 7, 1, 3, 5]
```

Find the smallest number in [2, 3, 7, 1, 3, 5], which is 1. Swap 1 with 2.

```
lst = [1, 3, 7, 2, 3, 5]
```

Find the smallest number in [3, 7, 2, 3, 5], which is 2. Swap 2 with 3.

```
lst = [1, 2, 7, 3, 3, 5]
```

Find the smallest number in [7, 3, 3, 5], which is 3. Swap 3 with 7.

```
lst = [1, 2, 3, 7, 3, 5]
```

Find the smallest number in [7, 3, 5], which is 3. Swap 3 with 7.

```
lst = [1, 2, 3, 3, 7, 5]
```

Find the smallest number in [7, 5], which is 5. Swap 5 with 7.

```
lst = [1, 2, 3, 3, 5, 7]
```

3.1 Implement the Selection Sort function.

Hint: You can use `lst.remove(val)` to remove the first `val` from `lst`

```
# TODO: Implement the Selection Sort function
def SelectionSort(lst):
    """Sorts lst using the selection sort algorithm.
    Arguments: lst (list)
    Returns: sorted list
    """
    pass # Remove this line and add your code here.
```

```
# Test cases
lst = [2, 3, 7, 1, 3, 5]
check(SelectionSort(lst), [1, 2, 3, 3, 5, 7])

lst = []
check(SelectionSort(lst), [])

lst = ["b", "aaa", "addis"]
check(SelectionSort(lst), ["aaa", "addis", "b"])
```

3.2 What is the time complexity of your Selection Sort implementation in Big-O notation?

3.3 Implement Selection Sort using recursion.

```
# TODO: Implement the recursive Selection Sort function
def RecSelectionSort(lst):
    """Sorts lst using the selection sort algorithm (recursive
    version).
    Arguments: lst (list), start (int)
    Returns: sorted list
    """

# Test cases
check(RecSelectionSort([2, 3, 7, 1, 3, 5]), [1, 2, 3, 3, 5, 7])
check(RecSelectionSort([]), [])
check(RecSelectionSort(["b", "aaa", "addis"]), ["aaa", "addis", "b"])
```

3.4 Implement Selection Sort with swapping. You can only use swapping between elements, i.e., your code should not define a new list and should not use `append`, `insert`, `list` additions, or deletions.

```
# TODO: Implement the Selection Sort function using swapping
def select_swap(L):
    """Sorts lst using the selection sort algorithm with swapping.
    Arguments: lst (list)
    Returns: sorted list
    """

    pass # Remove this line and add your code here

# Test cases
check(select_swap([2, 3, 7, 1, 3, 5]), [1, 2, 3, 3, 5, 7])
check(select_swap([]), [])
check(select_swap(["b", "aaa", "addis"]), ["aaa", "addis", "b"])
```

Exercise 4: Median Finding []

The **median** element of a list `L` is the element whose value is "right in the middle" of the list; it is smaller or equal to half of the elements in `L`.

For example:

- The median of [1, 2, 4, 5, 6] is 4.
- The median of [7, 5, 4, 1, 3, 2, 4, 9, 9, 7] is 5.

4.1 Write a function `median(L)` that takes a list `L` and returns the median of the list `L`.

```
# TODO: Implement the median finding function
def median(L):
    """Find the median in list L.
    Arguments: L (list of ints)
    Returns: (int or float)
    Effects: None
    """
    pass # Remove this line and write your solution here

# Test cases
L = [7, 5, 4, 1, 3, 2, 4, 9, 9, 7]
check(median(L), 4.5)

M = [4, 6, 78, 90, 12, 43, 9]
check(median(M), 12)
```

Exercise 5: Merge Lists [□□□]

5.1 Iterative Merge: Write an `iterative` function that, given two sorted lists, merges them into one sorted list.

```
# TODO: Implement the iterative merge function
def merge_iter(lst1, lst2):
    pass # remove this line and add your code here

# Test cases
check(merge_iter([2, 4, 5, 7, 8], [1, 3, 6, 9, 10]), [1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
check(merge_iter([], []), [])
```

5.2 What is the time complexity of your solution?

5.3 Recursive Merge Solve the above problem using recursion.

```
# TODO: Implement the recursive merge function
def merge_rec(lst1, lst2):
    pass # remove this line and add your code here

# Test cases
check(merge_rec([2, 4, 5, 7, 8], [1, 3, 6, 9, 10]), [1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
check(merge_rec([], []), [])
```

Exercise 6: Sort an Image [] [] [] []

Given a square canvas filled with random pixels, sort the pixels in each row according to the sum of their r, g, and b values.

Run the following cell first to define the required functions and to show how to generate a random image.

```
from simpleimage import SimpleImage
import random

def get_random_image(size):
    image = SimpleImage.blank(size, size)
    for y in range(size):
        for x in range(size):
            image.set_rgb(x, y, random.randint(0, 20),
random.randint(0, 255), random.randint(0, 20))
    return image

# Generate and display a random image
image = get_random_image(30)
image.show(resize_width=500)
```

6.1 Write a function `sortImg ( randImg )` that sorts the pixels in each row of the image according to the sum of their r, g, and b values.

```
# TODO: Implement the function to sort pixels in each row of the image
def sortImg(randImg):
    pass # remove this line and add your code

# Generate and display a new random image
image = get_random_image(30)
image.show(resize_width=500)

# Sort the image and display it
sortImg(image).show(resize_width=500)
```

Exercise 7: Counting Sort (Hard) [] [] [] []

7.1 Write a function that takes a list of **integers between 0 and 99** and sorts it. Can you solve it in linear time?

```
# TODO: Implement the Counting Sort function
def countSort(lst):
    pass # Remove this line and add your code

# Test case
import random
```

```
L = [random.randint(0, 99) for _ in range(10000)]  
check(countSort(L), sorted(L))
```

Congratulations 🎉